

OBJECT™
TECHNOLOGIES

Multithreading

www.object.co.in

What will be covered

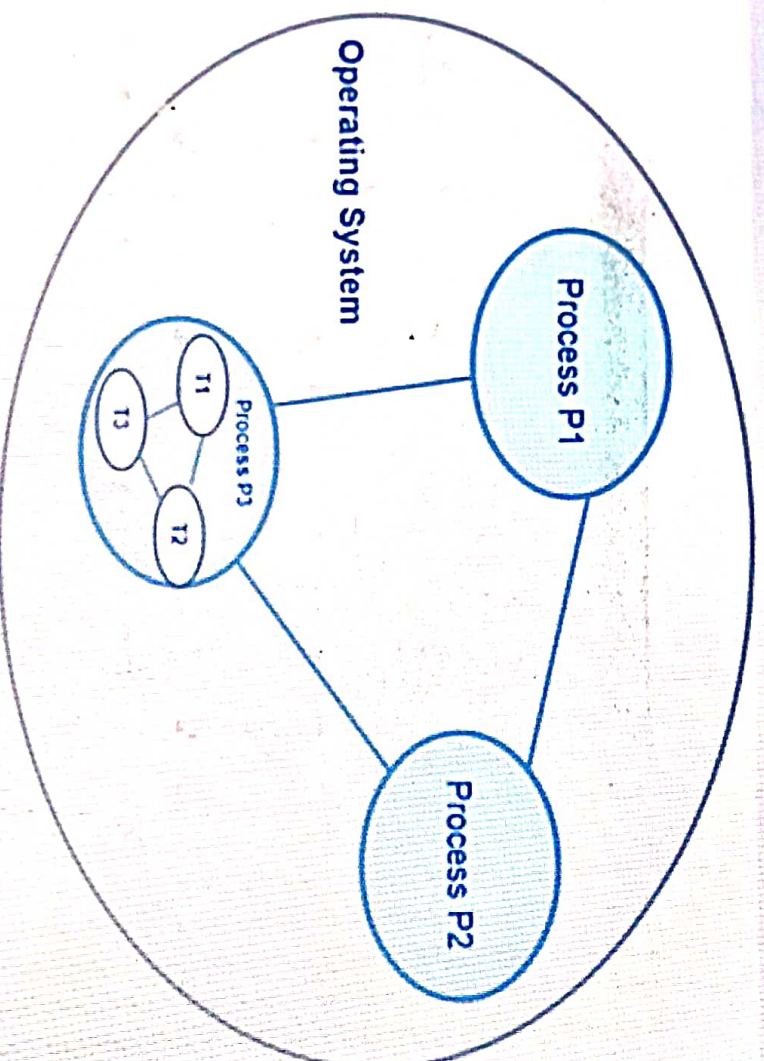
- Need of Multithreading
- What is multithreading
- What is thread
- Thread life cycle
- Ways of creating threads
- Creating multiple threads
- Thread priorities
- Use of join
- Thread synchronization
- Interthread communication

Need of Multithreading

- From the beginning, java applications are designed to run in distributed and networked environment. Console applications is a rare use of java.
- CPU is very fast and nowadays it even contains multiple cores, just one thread is not able to take advantage of all the cores, which means your costly hardware will remain idle for most of the time. By using multiple threads, you can take full advantage of multiple cores by serving more clients and serving them faster.
- One of the main reasons to use threads in Java is to make a task run parallel to another task.
- One of the most common scenarios where using multiple threads significantly improve an application's performance is a client-server application.
- Multi-threaded server means multiple clients can connect to the server at the same time. This means the next client doesn't have to wait until your application finish processing the request of the previous client.

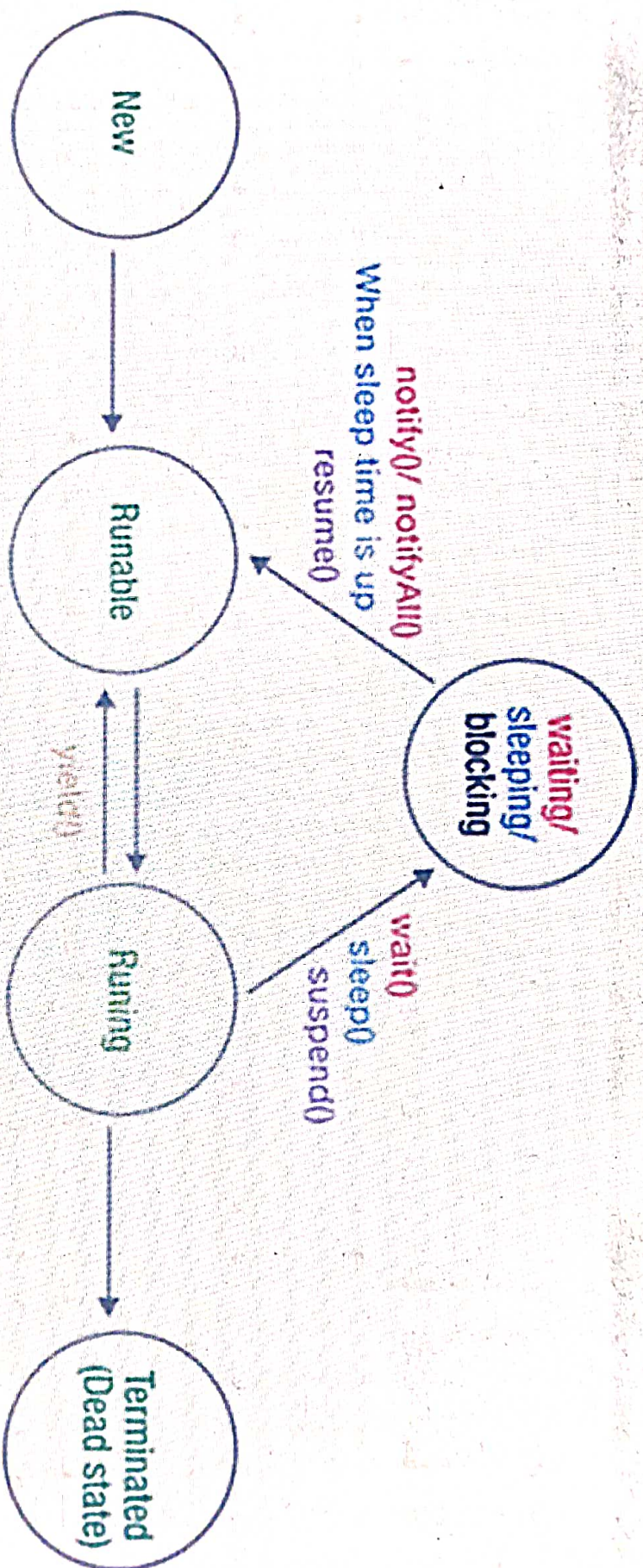
- **MULTITHREADING** in Java is a process of executing two or more threads simultaneously to maximum utilization of CPU.
- Multithreaded applications execute two or more threads run concurrently. Hence, it is also known as Concurrency in Java. Each thread runs parallel to each other.
- Multiple threads don't allocate separate memory area, hence they save memory.
- Multithreading and multiprocessing are different in nature.

What is a Thread



- A thread is a light-weight smallest part of a process that can run concurrently with the other parts (other threads) of the same process.
- it's an independent path of execution through program code
- Multiprocessing and multithreading, both are used to achieve multitasking.
- However, we use multithreading than multiprocessing because threads use a shared memory area. They don't allocate separate memory area so saves memory, and context-switching between the threads takes less time than process.
- If there occurs exception in one thread, it doesn't affect other threads.

Thread life cycle

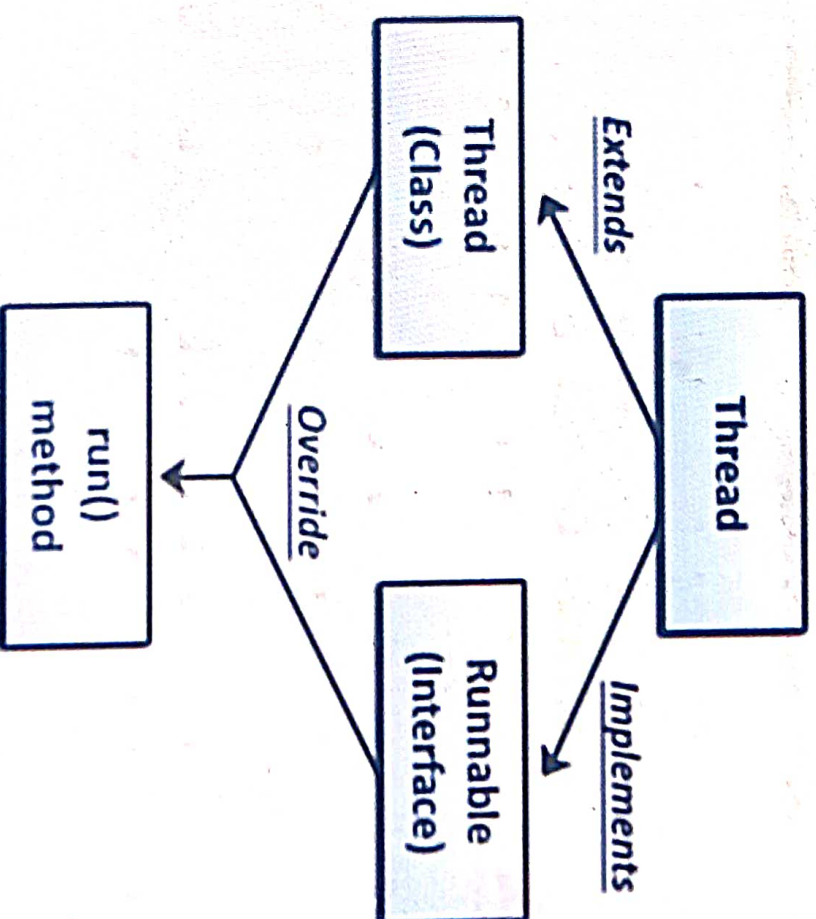


Understanding various states of a Java thread



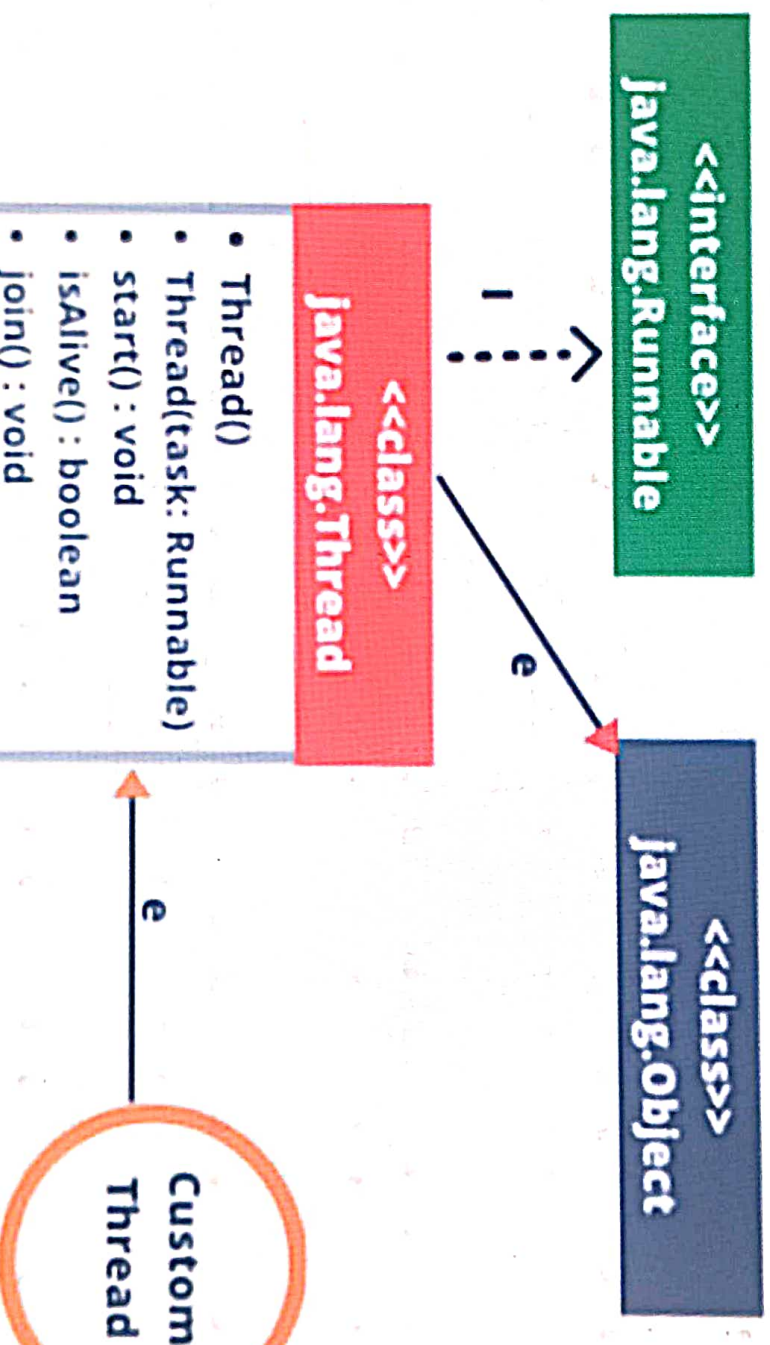
Ways of creating Threads

- Threads can be created by using two mechanisms :
 - Extending the Thread class
 - Implementing the Runnable Interface
- If the class is already extending from some other class, Thread class can not be used for creating thread. Only option is to use Runnable.
- Inheritance is mainly used for adding the functionality by extending the class. For creating thread, if we are only interested in providing the task to be done by the thread, we can make use of Runnable rather than Thread.



Creating thread by extending Thread class

- We can create thread by extending Thread class. Thread class provides run() method which needs to be overridden for specifying the task to be done by the thread.
- Thread class even provides few other methods which are like thread controller methods



Creating thread by extending Thread class

```
public class MyThread extends Thread
{
    public void run()
    {
        for(int i=1;i<=5;i++)
        {
            System.out.println("Child thread prints : "+i);
            try
            {
                Thread.sleep(500);
            }
            catch(InterruptedException e)
            {
                e.printStackTrace();
            }
        }
    }

    public static void main(String[] args)
    {
        Thread t=new MyThread();
        t.start();
    }
}
```



- Runnable is a single abstract method interface which has got only `run()` method.
- This method has to be overridden for defining the task to be performed by the thread.
- Thread created by implementing Runnable has to be dependent upon the Thread class for using thread controlling methods like `start()`.
- Runnable can provide only the method for the task to be performed by the thread.


```
public class MyTask implements Runnable
{
    public void run()
    {
        for(int i=1;i<=5;i++)
        {
            System.out.println("Child thread prints : "+i);
        }
        try
        {
            Thread.sleep(500);
        }
        catch(InterruptedException e)
        {
        }
    }
    public static void main(String[] args)
    {
        Runnable r = new MyTask();
        Thread t=new MyThread(r);
        t.start();
    }
}
```


- When a Java program starts up, one thread begins running immediately. This is usually called the main thread of our program, because it is the one that is executed when our program begins.
- Properties :
- It is the thread from which other “child” threads will be spawned.
- Often, it must be the last thread to finish execution because it performs various shutdown actions
- By default its priority is set to 5 (Normal priority)
- We can access the thread by using the method `Thread.currentThread()`
- Main thread is a user thread. The JVM will wait for any user thread to complete its task before terminating it.
- The threads spawned by main thread are also user threads by default.

- Java application can create as many threads as required by the application.
- All user threads will run in parallel with one another and finish the task assigned in multiple iterations of CPU time.
- When all user threads(non-daemon) finish the task, application gets terminated.
- Switching from one thread to another will be decided by operating system.
- Running the different tasks in parallel with the help of different threads helps in improving the performance



- Java application can create as many threads as required by the application.
- All user threads will run in parallel with one another and finish the task assigned in multiple iterations of CPU time.
- When all user threads(non-daemon) finish the task, application gets terminated.
- Switching from one thread to another will be decided by operating system.
- Running the different tasks in parallel with the help of different threads helps in improving the performance

Multithreading-I

Now as we are aware of the basic programming needs, handling runtime error, using IO library we will move further for learning very important API in java library. This API is used for creating multithreaded applications which is very important for real life applications. As java's purpose is to execute itself in networked, distributed environment, using multithreading becomes very important. In this chapter we will learn about what is a thread, creating threads and interrupting the thread by using sleep and join method.

Concept Of Thread

Need of multithreading :

Threading is a facility to allow multiple activities to coexist within a single process. Most modern operating systems support threads, Java is the first mainstream programming language to explicitly include threading within the language itself, rather than treating threading as a facility of the underlying operating system.

From the beginning, purpose of java applications is to execute in a networked and distributed environment. Application may get halted because of some reason like waiting for user input, waiting for some response from remote machine etc. In these scenarios CPU remains idle. Multithreading helps in simultaneous execution of two or more parts of a program to utilize the CPU time maximally. A multithreaded program contains two or more parts that can run concurrently. Idle time of CPU can be kept to minimum.

The process of executing multiple threads simultaneously is known as multithreading.

Benefits of multithreading :

1. Better use of system resources.
2. Parallel execution of tasks and thus less execution time
3. Enhanced performance on multi-processor machines
4. Improved GUI responsiveness
5. Independent threads (don't impact other threads of the same process if an exception occurs)

Concept of thread :

1. Threads are lightweight sub-processes,
2. The thread is the smallest unit of execution which has its separate path of execution
3. Multiple threads within a process share the same memory address space, which means they have access to the same variables and objects, and they allocate objects from the same heap

Note : Thread is nothing but finally a chunk of code which is assigned some task

Benefits of multithreading :

1. Better use of system resources.
2. Parallel execution of tasks and thus less execution time
3. Enhanced performance on multi-processor machines
4. Improved GUI responsiveness
5. Independent threads (don't impact other threads of the same process if an exception occurs)

Concept of thread :

1. Threads are lightweight sub-processes,
2. The thread is the smallest unit of execution which has its separate path of execution
3. Multiple threads within a process share the same memory address space, which means they have access to the same variables and objects, and they allocate objects from the same heap

Note : Thread is nothing but finally a chunk of code which is assigned some task

Thread Lifecycle

The thread goes through different states right from when it is created upto it finishes its task:

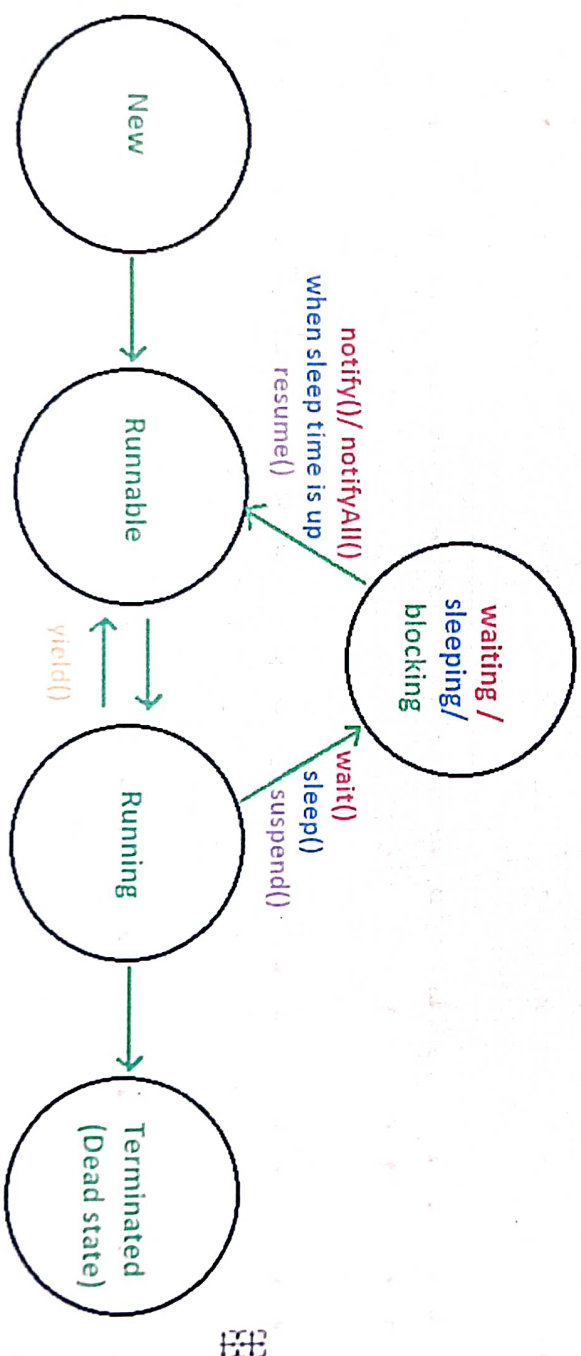


Fig. THREAD STATES

States through which the thread has to pass to complete its lifespan.

FIG. THREAD STATES

States through which the thread has to pass to complete its lifespan.

1. New
2. Runnable
3. Running
4. Non-Runnable (blocked)
5. Terminated

Brief of each life cycle phase :

1. New- A thread is in New state when you create the instance of thread class, but the start() method is yet to get called.
2. Runnable- The thread is in runnable after executing the start() method. In this stage, it waits for the thread scheduler to run it.
3. Running- A thread picked by the thread scheduler for execution remains in running state. Actual CPU time is allocated to the thread
4. Non-Runnable (blocked)- In this state, the thread remains alive, but it is not eligible to run. It may be due to some sleep operation, waiting for any File I/O to finish or in the locked state, etc.
5. Terminated- A thread is said to be in the terminated state when its Run() method exits which indicates that task of the thread has been completed

Note : In the above diagram focus on arrows. From runnable state thread will never come back to new state. Similarly thread will not enter from new to running state directly.

Creating Threads

There are two ways to create a thread in Java:

1. By extending Thread class.

Few methods of Thread class are listed below :

getName(): It is used for Obtaining a thread's name
getPriority(): Obtain a thread's priority

isAlive(): Determine if a thread is still running

join(): Wait for a thread to terminate

run(): Entry point for the thread

sleep(): suspend a thread for a period of time

Creating Threads

There are two ways to create a thread in Java:

1. By extending Thread class.

Few methods of Thread class are listed below :

getName(): It is used for Obtaining a thread's name

getPriority(): Obtain a thread's priority

isAlive(): Determine if a thread is still running

join(): Wait for a thread to terminate

run(): Entry point for the thread

sleep(): suspend a thread for a period of time

start(): start a thread by calling its run() method

It provides run() method which is used to implement the task to be performed by the thread. Rest of the methods are needed to control the thread execution.

Example :

```
public class MyThread extends Thread{
```

```
    public void run()
```

```
    {
```

```
        for(int i=1;i<=30;i++)
```

```
        {
```

```
            System.out.println("child thread prints : "+i);
```

```
        }
```

```
    }
```

```
}
```

Using above thread in the application :

```
public class MyThreadDemo
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        Thread t = new MyThread(); //creating thread instance, in new state
```

```
        t.start(); //thread converted to runnable state
```


Example :

```
public class MyThread extends Thread{
    public void run()
    {
        for(int i=1;i<=30;i++)
        {
            System.out.println("child thread prints : "+i);
        }
    }
}
```

Using above thread in the application :

```
public class MyThreadDemo
{
    public static void main(String[] args)
    {
        Thread t = new MyThread(); //creating thread instance, in new state
        t.start(); //thread converted to runnable state
        for(int i=1;i<=30;i++)
        {
            System.out.println("main thread prints : "+i);
        }
    }
}
```

In the above application, main thread creates child thread and then both the threads run concurrently. (note really concurrently but switching between one another)

2. By implementing Runnable interface.

This interface is single abstract method interface. It has only one method run() which is used to define the task to be performed by the thread. For other controlling methods of thread, Runnable depends on Thread.

Note: Design guidelines suggest, if no other method than the run() method in the thread class is reimplemented, it is better to create the thread by implementing Runnable interface.

2. By implementing Runnable interface.

This interface is single abstract method interface. It has only one method `run()` which is used to define the task to be performed by the thread. For other controlling methods of thread, `Runnable` depends on `Thread`.

Note: Design guidelines suggest, if no other method than the `run()` method in the thread class is reimplemented, it is better to create the thread by implementing `Runnable` interface.

Example :

```
public class MyTask implements Runnable
{
    public void run()
    {
        for(int i=1;i<=30;i++)
        {
            System.out.println("child thread prints : "+i);
        }
    }
}

public class MyTaskDemo
{
    public static void main(String[] args)
    {
        MyTask task = new MyTask();
        Thread th = new Thread(task); //creating a thread by one more constructor
        th.start();
        for(int i=1;i<=30;i++)
        {
            System.out.println("main thread prints : "+i);
        }
    }
}
```

Default thread :

1. When a Java program starts up, one thread begins running immediately. This is usually called the main thread of our

Default thread :

1. When a Java program starts up, one thread begins running immediately. This is usually called the main thread of our program, because it is the one that is executed when our program begins.
2. It is the thread from which other ?child? threads will be spawned.

3. Often, it must be the last thread to finish execution because it performs various shutdown actions

4. The main thread is created automatically when our program is started. To control it we must obtain a reference to it. This can be done by calling the method `currentThread()` which is present in `Thread` class.

`Thread cur_thread = Thread.currentThread();` //static method in `Thread` class

5. This method returns a reference to the thread on which it is called. The default priority of Main thread is 5 and for all remaining user threads priority will be inherited from parent to child.

User threads and daemon threads

In Java, there are two types of threads:

1. **Daemon Thread** : These type of threads are low priority threads which always run in background and user threads are high priority threads which always run in foreground.
2. **User Thread** : These type of thread or Non-Daemon are designed to do specific or complex task where as daemon threads are used to perform supporting tasks. JVM doesn't wait for daemon thread to finish but it waits for User Thread.

Use Of Sleep, Join

`sleep()` and `join()` both the methods are used for thread interruption.
`sleep()` method

1. `Thread.sleep` causes the current thread to suspend execution for a specified period.
2. This is an efficient means of making processor time available to the other threads of an application or other applications that might be running on a computer system.
3. `Thread.sleep()` method needs the time specified in milliseconds. The argument value for milliseconds can't be negative, else it throws `IllegalArgumentException`. One more overloaded sleep version is available which accepts time in milliseconds and nanoseconds
4. The actual time thread sleeps before waking up and start execution depends on system timers and schedulers. For a quiet system, the actual time for sleep is near to the specified sleep time but for a busy system it will be little bit more.
5. Thread sleep doesn't release the lock on current thread


```

public class TableThread extends Thread
{
    int num;
    public TableThread(int num) {
        super();
        this.num = num;
    }
    public void run()
    {
        for(int i=1;i<=10;i++)
        {
            System.out.println(num + "*" + i + " = " + (num*i));
        }
        try
        {
            Thread.sleep(500);
        } catch (InterruptedException e)
        {
            e.printStackTrace();
        }
    }
}

```

Above thread prints the table of a specified number by sleeping for 500 ms before every next iteration.

Note : This method throws InterruptedException which is checked exception so it needs to be handled.

join() method

1. The join() method is used to hold the execution of currently running thread until the specified thread is dead/finished execution).
2. A join() is a final method of Thread class
3. It can even be used to join the start of a thread's execution to the end of another thread's execution so that a thread will not start running until another thread has ended.
4. join() method has got overloaded versions which accepts even the time in milliseconds.

```

JoinTest t1 = new JoinTest();
JoinTest t2 = new JoinTest();

```


4. join() method has got overloaded versions which accepts even the time in milliseconds.

```
JoinTest t1 = new JoinTest();
JoinTest t2 = new JoinTest();
JoinTest t3 = new JoinTest();
t1.start();
try {
    t1.join(); // calling join() method
}
catch(Exception e)
{
    System.out.println(e);
}
t2.start();
t3.start();
```

In this case join() method is called on t1 by main thread. So main thread will not start running until t1 thread finishes.

Note :

join() and sleep() both the methods throw InterruptedException.

sleep() is a static method in Thread class whereas join() is a non-static method.

Assignments

1. Show the name and priority of default thread that gets created in any java application.
2. Create a thread for just printing numbers from 1 to 5. Make the thread sleep in between for 500 ms. Create the instance from main thread.(Use Thread class as well as Runnable interface for creating thread)
3. Repeat above assignment for declaring Thread controller within the the class implementing Runnable.
4. Create a thread for counting down the number from a particular number to zero. Create 3 thread instances from main thread and make the main thread stop after all the thread executes completely.
5. Create a thread for copying the contents from source file to destination file. Accept source file name and destination file name from the user in main thread and pass it to other thread.
6. Create a thread for printing prime numbers in the given range. Accept the start and end number from user in main thread and pass it to the child thread.

6. Create a thread for printing prime numbers in the given range. Accept the start and end number from user in main thread and pass it to the child thread.

Assignments-1

1. Show the name and priority of default thread that gets created in any java application.
2. Create a thread for just printing numbers from 1 to 5. Make the thread sleep in between for 500 ms. Create the instance from main thread.(Use Thread class as well as Runnable interface for creating thread)
3. Create a thread for counting down the number from a particular number to zero. Create 3 thread instances from main thread and make the main thread stop after all the thread executes completely.
4. Create a thread for copying the contents from source file to destination file. Accept source file name and destination file name from the user in main thread and pass it to other thread.

Assignments-2

1. Show the name and priority of default thread that gets created in any java application.
2. Create a thread for just printing numbers from 1 to 5. Make the thread sleep in between for 500 ms. Create the instance from main thread.(Use Thread class as well as Runnable interface for creating thread)
3. Create a thread for counting down the number from a particular number to zero. Create 3 thread instances from main thread and make the main thread stop after all the thread executes completely.
4. Create a thread for copying the contents from source file to destination file. Accept source file name and destination file name from the user in main thread and pass it to other thread.

Important Questions for: Multithreading-I

Interview Questions

what is local thread

Coding Questions

write a program to find nth max element in array

program to check number and string is palindrome or not

program to find wordcount,spacecount and digitcount in given string